

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу
«Операционные системы»

Группа: М8О-215Б-23

Студент: Аралов И.А.

Преподаватель: Миронов Е.С.

Оценка: _____

Дата: 03.03.24

Москва, 2024

Постановка задачи

Вариант 1.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решения задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или через отображаемые файлы (memory-mapped files). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Общий метод и алгоритм решения

Системные вызовы:

- `fork()`: Создает дочерние процессы.
- `waitpid()`: Ожидает завершения дочерних процессов.
- `mmap()`: Отображает файл shared memory в память.
- `munmap()`: Освобождает отображенную память.
- `open()`: Открывает файл shared memory.
- `close()`: Закрывает файловый дескриптор.
- `ftruncate()`: Устанавливает размер файла shared memory.
- `sem_init()`: Инициализирует семафоры.
- `sem_destroy()`: Уничтожает семафоры.
- `sem_post()`: Увеличивает значение семафора.
- `sem_wait()`: Уменьшает значение семафора.

Алгоритм работы программы

1. Инициализация (Родительский процесс)

1. Создание разделяемой памяти:

- Вызов `shm_open("/my_shared_memory", O_CREAT | O_RDWR, 0666)` создает объект разделяемой памяти.
- Установка размера памяти через `ftruncate(fd, shared_size)`.
- Отображение памяти в адресное пространство процесса через `mmap`.

2. Создание семафоров:

- Создание семафоров `sem_parent` и `sem_child` через `sem_open` с флагом `O_CREAT`.
- Инициализация семафоров: `sem_parent = 0`, `sem_child = 0`.

3. Создание дочернего процесса:

- Вызов `fork()` для создания дочернего процесса.
- В дочернем процессе выполняется `execl("./child", "child", NULL)` для запуска программы `child`.

2. Работа родительского процесса

1. Ввод данных:

- Родительский процесс запрашивает у пользователя число и сохраняет его в `shared_data->number`.

2. Уведомление дочернего процесса:

- Родительский процесс вызывает `sem_post(sem_child)`, чтобы разблокировать дочерний процесс.

3. Ожидание результата:

- Родительский процесс вызывает `sem_wait(sem_parent)`, чтобы дождаться завершения работы дочернего процесса.

4. Вывод результата:

- Родительский процесс читает результат из `shared_data->message` и выводит его на экран.

5. Завершение работы:

- Родительский процесс освобождает ресурсы:
 - `munmap` для отключения разделяемой памяти.
 - `shm_unlink` для удаления объекта разделяемой памяти.
 - `sem_close` и `sem_unlink` для закрытия и удаления семафоров.

3. Работа дочернего процесса

1. Открытие разделяемой памяти:

- Дочерний процесс открывает существующий объект разделяемой памяти через `shm_open("/my_shared_memory", O_RDWR, 0666)`.
- Отображает память через `mmap`.

2. Открытие семафоров:

- Дочерний процесс открывает существующие семафоры через `sem_open("/sem_parent", 0)` и `sem_open("/sem_child", 0)`.

3. Ожидание данных от родителя:

- Дочерний процесс вызывает `sem_wait(sem_child)`, чтобы дождаться уведомления от родителя.

4. Обработка данных:

- Дочерний процесс читает число из `shared_data->number`.
- Проверяет, является ли число отрицательным, простым или составным.
- Записывает результат в `shared_data->message`.

5. Уведомление родителя:

- Дочерний процесс вызывает `sem_post(sem_parent)`, чтобы уведомить родителя о завершении работы.

6. Завершение работы:

- Дочерний процесс освобождает ресурсы:
 - `mmap` для отключения разделяемой памяти.
-

4. Схема взаимодействия

1. Родительский процесс:

- Создает ресурсы (память, семафоры).
- Записывает данные в разделяемую память.
- Уведомляет дочерний процесс через `sem_post(sem_child)`.
- Ожидает результат через `sem_wait(sem_parent)`.

2. Дочерний процесс:

- Ожидает данные через `sem_wait(sem_child)`.
- Обработывает данные и записывает результат в разделяемую память.
- Уведомляет родителя через `sem_post(sem_parent)`.

Код программы

parent.c

```
#include <stdio.h>

#include <stdlib.h>

#include <sys/mman.h>

#include <fcntl.h>

#include <semaphore.h>

#include <sys/wait.h>

#include <unistd.h>

#include <string.h>

#define MAX_SIZE 100

struct SharedData {

    int numbers[MAX_SIZE];

    char message[256];

};

int main() {

    // Размер общей памяти

    size_t shared_size = sizeof(struct SharedData);

    // Создание разделяемой памяти

    int fd = open("/my_shared_memory", O_CREAT | O_RDWR, 0666); //

    if (fd == -1) {

        perror("Ошибка при создании разделяемой памяти");

        return 1;

    }

    ftruncate(fd, shared_size);
```

```
// Отображение памяти

struct SharedData* shared_data = (struct SharedData*)mmap(NULL, shared_size,
PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

if (shared_data == MAP_FAILED) {

    perror("Ошибка при отображении разделяемой памяти");

    return 1;

}

close(fd);


// Создание семафоров

sem_t* sem_parent = sem_open("/sem_parent", O_CREAT, 0666, 0);
sem_t* sem_child = sem_open("/sem_child", O_CREAT, 0666, 0);
if (sem_parent == SEM_FAILED || sem_child == SEM_FAILED) {

    perror("Ошибка при создании семафоров");

    return 1;

}


// Создание дочернего процесса

pid_t pid = fork();

if (pid < 0) {

    perror("Ошибка при создании дочернего процесса");

    return 1;

}

if (pid > 0) { // Родительский процесс

    int number, numbers[MAX_SIZE];

    int size = 0;
```

```
char buffer[1024]; // Буфер для чтения строки
```

```
printf("Введите данные в формате: «число число число<newline>».\n");
```

```
if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
```

```
    fprintf(stderr, "Ошибка чтения ввода\n");
```

```
    return 1;
```

```
}
```

```
char *token = strtok(buffer, " "); // Разделение строки на токены
```

```
while (token != NULL && size < MAX_SIZE) {
```

```
    numbers[++size] = atoi(token); // Преобразование токена в число и  
сохранение в массив
```

```
    token = strtok(NULL, " ");
```

```
}
```

```
numbers[0] = size;
```

```
// Запись чисел в разделяемую память
```

```
memcpy(shared_data->numbers, numbers, sizeof(numbers));
```

```
sem_post(sem_child); // Уведомляем дочерний процесс
```

```
// Ожидание результата от дочернего процесса
```

```
sem_wait(sem_parent);
```

```
printf("Результат: %s\n", shared_data->message);
```

```
// Завершаем работу
```

```
waitpid(pid, NULL, 0);
```

```

        // Удаление ресурсов

        munmap(shared_data, shared_size);

        sem_close(sem_parent);

        sem_close(sem_child);

        sem_unlink("/sem_parent");

        sem_unlink("/sem_child");

    } else { // Дочерний процесс

        execl("./child.o", "child", NULL);

        perror("Ошибка при вызове дочернего процесса");

        return 1;

    }

    return 0;

}

```

child.c

```

#include <stdio.h>

#include <stdlib.h>

#include <sys/mman.h>

#include <fcntl.h>

#include <semaphore.h>

#include <unistd.h>

#include <math.h>

#include <string.h>

#define MAX_SIZE 100

struct SharedData {

```



```
int numbers[MAX_SIZE];

char message[256];

};

int sum_array(int *array, int size) {

    int sum = 0;

    for (int i = 0; i < size; i++) {

        sum += array[i + 1];

    }

    return sum;

}

int main() {

    int numbers[MAX_SIZE];

    // Открытие разделяемой памяти

    int fd = open("/my_shared_memory", O_RDWR, 0666);

    if (fd == -1) {

        perror("Ошибка при открытии разделяемой памяти");

        return 1;

    }

    // Отображение памяти

    size_t shared_size = sizeof(struct SharedData);

    struct SharedData* shared_data = (struct SharedData*)mmap(NULL, shared_size,
PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

    if (shared_data == MAP_FAILED) {
```

```
        perror("Ошибка при отображении разделяемой памяти");

        return 1;
    }

    close(fd);

    // Открытие семафоров

    sem_t* sem_parent = sem_open("/sem_parent", 0);

    sem_t* sem_child = sem_open("/sem_child", 0);

    if (sem_parent == SEM_FAILED || sem_child == SEM_FAILED) {

        perror("Ошибка при открытии семафоров");

        return 1;
    }

    // Ожидание чисел от родительского процесса

    sem_wait(sem_child);

    memcpy(numbers, shared_data->numbers, sizeof(shared_data->numbers));

    int sum = sum_array(numbers, numbers[0]);

    int file = open("result.txt", O_WRONLY | O_CREAT | O_TRUNC, 0666);

    if (file == -1) {

        strncpy(shared_data->message, "Ошибка при открытии файла",
sizeof(shared_data->message));

        sem_post(sem_parent);

        munmap(shared_data, shared_size);

        return 1;
    }
}
```

```
}

// Форматирование результата в строку

char buffer[16];

int n = snprintf(buffer, sizeof(buffer), "%d\n", sum);

if (n <= 0) {

    strncpy(shared_data->message, "Ошибка при форматировании строки",
sizeof(shared_data->message));

    close(file);

    sem_post(sem_parent);

    munmap(shared_data, shared_size);

    return 1;

}

// Запись строки в файл

if (write(file, buffer, n) == -1) {

    strncpy(shared_data->message, "Ошибка записи в файл",
sizeof(shared_data->message));

    close(file);

    sem_post(sem_parent);

    munmap(shared_data, shared_size);

    return 1;

}

    strncpy(shared_data->message, "Сумма записана в файл",
sizeof(shared_data->message));

    close(file);

    sem_post(sem_parent);

    munmap(shared_data, shared_size);
```

```
    return 0;

}
```

Протокол работы программы

Тестирование:

```
root@fa7703109f3c:/workspaces/OS_labs/lab1/src# ./parent
Введите данные в формате: «число число число<newline>».
1 2 3 4 5
Программа завершила работу без ошибок
root@fa7703109f3c:/workspaces/OS_labs/lab1/src# cat result.txt
15
```

Strace:

```
root@fa7703109f3c:/workspaces/OS_labs/lab3/src# strace -f ./parent.o
execve("./parent.o", [ "./parent.o" ], 0x7ffdb9123a48 /* 30 vars */) = 0
brk(NULL)                               = 0xaf9000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f04bfaf0000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=25258, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 25258, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f04bfae9000
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\20t\2\0\0\0\0\0"... , 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... , 784, 64)
= 784
newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=1922136, ...}, AT_EMPTY_PATH) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... , 784, 64)
= 784
mmap(NULL, 1970000, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f04bf908000
mmap(0x7f04bf92e000, 1396736, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x26000) = 0x7f04bf92e000
mmap(0x7f04bfa83000, 339968, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x17b000) = 0x7f04bfa83000
```

[illegible]

```

    openat(AT_FDCWD, "/dev/shm/sem.sem_child", O_RDWR|O_NOFOLLOW) = -1 ENOENT (No such file
or directory)

    getRandom("\x1c\xb8\x82\xa5\x01\x13\x3d\x70", 8, GRND_NONBLOCK) = 8

    newfstatat(AT_FDCWD, "/dev/shm/sem.uM3BBt", 0x7ffe8b5db410, AT_SYMLINK_NOFOLLOW) = -1
ENOENT (No such file or directory)

    openat(AT_FDCWD, "/dev/shm/sem.uM3BBt", O_RDWR|O_CREAT|O_EXCL, 0666) = 3

    write(3, "\0\0\0\0\0\0\0\0\200\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0", 32) = 32

    mmap(NULL, 32, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0) = 0x7f04bfaed000

    link("/dev/shm/sem.uM3BBt", "/dev/shm/sem.sem_child") = 0

    newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=32, ...}, AT_EMPTY_PATH) = 0

    unlink("/dev/shm/sem.uM3BBt") = 0

    close(3) = 0

    clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLDstrace:
Process 87504 attached

, child_tidptr=0x7f04bf905a10) = 87504

[pid 87504] set_robust_list(0x7f04bf905a20, 24 <unfinished ...>

[pid 87501] newfstatat(1, "", <unfinished ...>

[pid 87504] <... set_robust_list resumed>) = 0

[pid 87501] <... newfstatat resumed>{st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0),
...}, AT_EMPTY_PATH) = 0

[pid 87501] write(1, "\320\222\320\262\320\265\320\264\320\270\321\202\320\265
\320\264\320\260\320\275\320\275\321\213\320\265 \320\262 \321"..., 94 <unfinished ...>

Введите данные в формате: «число число число<newline>».

[pid 87504] execve("./child.o", ["child"], 0x7ffe8b5dbe78 /* 30 vars */ <unfinished
...>

[pid 87501] <... write resumed>) = 94

[pid 87501] newfstatat(0, "", {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...},
AT_EMPTY_PATH) = 0

[pid 87501] read(0, <unfinished ...>

[pid 87504] <... execve resumed>) = 0

[pid 87504] brk(NULL) = 0x11fd000

[pid 87504] mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f5b4fa87000

[pid 87504] access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)

[pid 87504] openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3

[pid 87504] newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=25258, ...},
AT_EMPTY_PATH) = 0

```

```

[pid 87504] mmap(NULL, 25258, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f5b4fa80000

[pid 87504] close(3) = 0

[pid 87504] openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3

[pid 87504] read(3,
"\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\20t\2\0\0\0\0"... , 832) = 832

[pid 87504] pread64(3,
"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... , 784, 64) = 784

[pid 87504] newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=1922136, ...},
AT_EMPTY_PATH) = 0

[pid 87504] pread64(3,
"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... , 784, 64) = 784

[pid 87504] mmap(NULL, 1970000, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x7f5b4f89f000

[pid 87504] mmap(0x7f5b4f8c5000, 1396736, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x26000) = 0x7f5b4f8c5000

[pid 87504] mmap(0x7f5b4fa1a000, 339968, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x17b000) = 0x7f5b4fa1a000

[pid 87504] mmap(0x7f5b4fa6d000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1ce000) = 0x7f5b4fa6d000

[pid 87504] mmap(0x7f5b4fa73000, 53072, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f5b4fa73000

[pid 87504] close(3) = 0

[pid 87504] mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f5b4f89c000

[pid 87504] arch_prctl(ARCH_SET_FS, 0x7f5b4f89c740) = 0

[pid 87504] set_tid_address(0x7f5b4f89ca10) = 87504

[pid 87504] set_robust_list(0x7f5b4f89ca20, 24) = 0

[pid 87504] rseq(0x7f5b4f89d060, 0x20, 0, 0x53053053) = 0

[pid 87504] mprotect(0x7f5b4fa6d000, 16384, PROT_READ) = 0

[pid 87504] mprotect(0x403000, 4096, PROT_READ) = 0

[pid 87504] mprotect(0x7f5b4fab9000, 8192, PROT_READ) = 0

[pid 87504] prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
rlim_max=RLIM64_INFINITY}) = 0

[pid 87504] munmap(0x7f5b4fa80000, 25258) = 0

[pid 87504] openat(AT_FDCWD, "/my_shared_memory", O_RDWR) = 3

[pid 87504] mmap(NULL, 656, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0) = 0x7f5b4fa86000

[pid 87504] close(3) = 0

```

```

[pid 87504] openat(AT_FDCWD, "/dev/shm/sem.sem_parent", O_RDWR|O_NOFOLLOW) = 3
[pid 87504] newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=32, ...}, AT_EMPTY_PATH) =
0

[pid 87504] getrandom("\x48\x27\x2b\x02\xb6\x69\x51\x04", 8, GRND_NONBLOCK) = 8
[pid 87504] brk(NULL) = 0x11fd000
[pid 87504] brk(0x121e000) = 0x121e000
[pid 87504] mmap(NULL, 32, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0) = 0x7f5b4fa85000
[pid 87504] close(3) = 0
[pid 87504] openat(AT_FDCWD, "/dev/shm/sem.sem_child", O_RDWR|O_NOFOLLOW) = 3
[pid 87504] newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=32, ...}, AT_EMPTY_PATH) =
0

[pid 87504] mmap(NULL, 32, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0) = 0x7f5b4fa84000
[pid 87504] close(3) = 0
[pid 87504] futex(0x7f5b4fa84000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL,
FUTEX_BITSET_MATCH_ANY <unfinished ...>

[pid 87501] <... read resumed>0xaf9770, 1024) = ? ERESTARTSYS (To be restarted if
SA_RESTART is set)

[pid 87504] <... futex resumed> = ? ERESTARTSYS (To be restarted if SA_RESTART
is set)

[pid 87501] --- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
[pid 87504] --- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
[pid 87501] read(0, <unfinished ...>

[pid 87504] futex(0x7f5b4fa84000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL,
FUTEX_BITSET_MATCH_ANY) = ? ERESTARTSYS (To be restarted if SA_RESTART is set)

[pid 87501] <... read resumed>0xaf9770, 1024) = ? ERESTARTSYS (To be restarted if
SA_RESTART is set)

[pid 87504] --- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
[pid 87501] --- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
[pid 87504] futex(0x7f5b4fa84000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL,
FUTEX_BITSET_MATCH_ANY <unfinished ...>

[pid 87501] read(0, 0xaf9770, 1024) = ? ERESTARTSYS (To be restarted if SA_RESTART
is set)

[pid 87504] <... futex resumed> = ? ERESTARTSYS (To be restarted if SA_RESTART
is set)

[pid 87501] --- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
[pid 87504] --- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
[pid 87501] read(0, <unfinished ...>

```


[pid 87504] futex(0x7f5b4fa84000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL, FUTEX_BITSET_MATCH_ANY <unfinished ...>

[pid 87501] <... read resumed>0xaf9770, 1024) = ? ERESTARTSYS (To be restarted if SA_RESTART is set)

[pid 87504] <... futex resumed> = ? ERESTARTSYS (To be restarted if SA_RESTART is set)

[pid 87501] --- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---

[pid 87504] --- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---

[pid 87501] read(0, <unfinished ...>

[pid 87504] futex(0x7f5b4fa84000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL, FUTEX_BITSET_MATCH_ANY <unfinished ...>

[pid 87501] <... read resumed>0xaf9770, 1024) = ? ERESTARTSYS (To be restarted if SA_RESTART is set)

[pid 87504] <... futex resumed> = ? ERESTARTSYS (To be restarted if SA_RESTART is set)

[pid 87501] --- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---

[pid 87504] --- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---

[pid 87501] read(0, <unfinished ...>

[pid 87504] futex(0x7f5b4fa84000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL, FUTEX_BITSET_MATCH_ANY <unfinished ...>

[pid 87501] <... read resumed>0xaf9770, 1024) = ? ERESTARTSYS (To be restarted if SA_RESTART is set)

[pid 87504] <... futex resumed> = ? ERESTARTSYS (To be restarted if SA_RESTART is set)

[pid 87501] --- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---

[pid 87504] --- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---

[pid 87501] read(0, <unfinished ...>

[pid 87504] futex(0x7f5b4fa84000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL, FUTEX_BITSET_MATCH_ANY <unfinished ...>

[pid 87501] <... read resumed>0xaf9770, 1024) = ? ERESTARTSYS (To be restarted if SA_RESTART is set)

[pid 87504] <... futex resumed> = ? ERESTARTSYS (To be restarted if SA_RESTART is set)

[pid 87501] --- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---

[pid 87504] --- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---

[pid 87501] read(0, <unfinished ...>

[pid 87504] futex(0x7f5b4fa84000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL, FUTEX_BITSET_MATCH_ANY <unfinished ...>

```

[pid 87501] <... read resumed>0xaf9770, 1024) = ? ERESTARTSYS (To be restarted if
SA_RESTART is set)

[pid 87504] <... futex resumed>) = ? ERESTARTSYS (To be restarted if SA_RESTART
is set)

[pid 87501] --- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
[pid 87504] --- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
[pid 87501] read(0, <unfinished ...>

[pid 87504] futex(0x7f5b4fa84000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL,
FUTEX_BITSET_MATCH_ANY <unfinished ...>

[pid 87501] <... read resumed>0xaf9770, 1024) = ? ERESTARTSYS (To be restarted if
SA_RESTART is set)

[pid 87504] <... futex resumed>) = ? ERESTARTSYS (To be restarted if SA_RESTART
is set)

[pid 87501] --- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
[pid 87504] --- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
[pid 87501] read(0, <unfinished ...>

[pid 87504] futex(0x7f5b4fa84000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL,
FUTEX_BITSET_MATCH_ANY1 2 3 4 5
<unfinished ...>

[pid 87501] <... read resumed>"1 2 3 4 5\n", 1024) = 10
[pid 87501] futex(0x7f04bfaed000, FUTEX_WAKE, 1) = 1
[pid 87504] <... futex resumed>) = 0

[pid 87501] futex(0x7f04bfaee000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL,
FUTEX_BITSET_MATCH_ANY <unfinished ...>

[pid 87504] openat(AT_FDCWD, "result.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
[pid 87504] write(3, "15\n", 3) = 3
[pid 87504] close(3) = 0
[pid 87504] futex(0x7f5b4fa85000, FUTEX_WAKE, 1 <unfinished ...>
[pid 87501] <... futex resumed>) = 0
[pid 87504] <... futex resumed>) = 1

[pid 87501] write(1,
"\320\240\320\265\320\267\321\203\320\273\321\214\321\202\320\260\321\202:
\320\241\321\203\320\274\320\274\320\260 \320"... , 60 <unfinished ...>

Результат: Сумма записана в файл

[pid 87504] munmap(0x7f5b4fa86000, 656 <unfinished ...>

[pid 87501] <... write resumed>) = 60
[pid 87504] <... munmap resumed>) = 0

```

```

[pid 87501] wait4(87504, <unfinished ...>

[pid 87504] exit_group(0)                = ?

[pid 87504] +++ exited with 0 +++

<... wait4 resumed>NULL, 0, NULL)        = 87504

--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=87504, si_uid=0, si_status=0,
si_utime=0, si_stime=1 /* 0.01 s */} ---

munmap(0x7f04bfaef000, 656)                = 0

munmap(0x7f04bfaee000, 32)                = 0

munmap(0x7f04bfaed000, 32)                = 0

unlink("/dev/shm/sem.sem_parent")         = 0

unlink("/dev/shm/sem.sem_child")          = 0

exit_group(0)                            = ?

+++ exited with 0 +++

```

Вывод

Лабораторная работа демонстрирует использование разделяемой памяти (shared memory) и семафоров для взаимодействия между процессами в Linux. Программа корректно обрабатывает ввод пользователя, синхронизирует работу процессов и завершается без утечек ресурсов.